Introduction

Welcome to the Healthcare Dashboard DAX Documentation. This guide is written for beginners — people who may be new to Power BI or DAX (Data Analysis Expressions). Every formula used in building the healthcare dashboard is explained here in plain English, with real-world examples so you can understand not just what the formula does, but why it exists and when to use it.

What is DAX?

DAX stands for Data Analysis Expressions. It is the formula language used in Microsoft Power BI, Excel Power Pivot, and SQL Server Analysis Services (SSAS). Think of DAX as the Excel formulas equivalent for Power BI — just more powerful. DAX can work with entire tables, filter data on the fly, and compute complex calculations like Year-over-Year changes.

What is this Dashboard about?

This Healthcare Dashboard tracks patient admissions, hospital billing, doctor performance, room utilization, and medical test results. The DAX formulas we built power every number, chart, and KPI card you see on that dashboard.

SECTION 1: CALENDAR TABLE

1.1 Calendar Table — The Date Foundation

Before we can analyze data by month, quarter, or year, Power BI needs a proper calendar table. A calendar table is a special helper table that contains one row for every single date in your dataset. All date-based calculations (monthly trends, year-over-year comparisons) depend on this table.

The Formula

```
Calendar =

var mindate = MIN(HealthData[Date of Admission])

var maxdate = MAX(HealthData[Date of Admission])

VAR BaseCalendar =

    CALENDAR(mindate, maxdate)

RETURN

ADDCOLUMNS(

    BaseCalendar,

    "Year", YEAR([Date]),

    "Quarter", QUARTER([Date]),

    "Month No", MONTH([Date]),

    "Week Num", WEEKNUM([Date]),

    "Week Day", WEEKDAY([Date]),
```

```
    "Day", DAY([Date]),
    "Month Name", FORMAT([Date], "MMMM"),
    "Month Short", FORMAT([Date], "MMM"),
    "Week", FORMAT([Date], "dddd"),
    "Quarter Number", "Q" & QUARTER([Date]),
    "Year Month", FORMAT([Date], "YYYY MMMM"),
    "Day Type", IF(WEEKDAY([Date], 2) > 5, "Weekend", "Weekday"),
    "Weekending", [Date] + (7 - WEEKDAY([Date]))
)
```

Step-by-Step Explanation

Part	What It Does
`MIN(HealthData[Date of Admission])`	Finds the earliest admission date in your data — this is where the calendar STARTS.
`MAX(HealthData[Date of Admission])`	Finds the latest admission date in your data — this is where the calendar ENDS.
`CALENDAR(mindate, maxdate)`	Creates a table with one row per day between the start and end date.
`ADDCOLUMNS(...)`	Adds extra columns to that date table, like Year, Month, Quarter, etc.
`YEAR([Date])`	Extracts the year number (e.g., 2023) from each date.

`MONTH([Date])`	Extracts the month number (1 = January, 12 = December).
`QUARTER([Date])`	Returns 1, 2, 3, or 4 — which quarter of the year.
`FORMAT([Date], "MMMM")`	Converts a date to a full month name like 'January' or 'February'.
`FORMAT([Date], "MMM")`	Short version of month name: 'Jan', 'Feb', etc.
`WEEKDAY([Date], 2) > 5`	Checks if the day is a weekend. Mode 2 means week starts Monday, so 6=Saturday, 7=Sunday.
`"Q" & QUARTER([Date])`	Combines the letter "Q" with the quarter number to create labels like Q1, Q2, Q3, Q4.
`[Date] + (7 - WEEKDAY([Date]))`	Calculates the date of the next Sunday — the week-ending date.

Real-World Example

Imagine you want to show "Total Patients admitted in Q3 2023" on your dashboard.

Without a Calendar Table, Power BI has no easy way to know which dates belong to Q3.

The Calendar Table acts like a reference index — every date in your patient data gets matched to a row in the Calendar Table, letting you filter and group by Quarter, Month, Year, etc.

SECTION 2: KEY PERFORMANCE INDICATORS (KPIs)

KPIs are the headline numbers that appear in cards at the top of most dashboards. Each formula below produces one of those summary numbers.

2.1 Admitted Patients

```
Admitted Patients = DISTINCTCOUNT(HealthData[Patient ID])
```

Why This Formula?

DISTINCTCOUNT counts unique values. If the same patient was admitted 3 times, DISTINCTCOUNT counts them as 1 patient. This avoids double-counting and gives you the true number of unique patients.

Syntax

DISTINCTCOUNT(<column>) — counts the number of distinct (unique) values in the specified column.

Real-Life Case Study

Imagine Patient John Doe (ID: P001) was admitted on 3 different dates. COUNT would return 3, but DISTINCTCOUNT returns 1. If your hospital wants to know how many unique individuals received care, DISTINCTCOUNT is the right choice.

2.2 Average Age

```
Average age = AVERAGE(HealthData[Age])
```

Why This Formula?

AVERAGE calculates the arithmetic mean of all values in a column. It adds up all patient ages and divides by the count of patients. This tells hospital management what the typical patient age looks like — useful for planning services (e.g., if average age is 65+, geriatric care becomes a priority).

Syntax

AVERAGE(<column>) — returns the arithmetic mean of all numbers in a column.

Real-Life Case Study

If your hospital has 3 patients aged 30, 50, and 70, AVERAGE returns (30+50+70)/3 = 50. This tells the hospital the average patient is 50 years old, helping plan appropriate treatments, dietary needs, and staffing.

2.3 Average Billing Amount

```
Avg Billing Amt = AVERAGE(HealthData[Billing Amount])
```

This shows the average bill per patient. Finance teams use this to understand revenue per patient and set pricing strategies. If the average billing amount drops over time, it could signal shorter stays or lower-cost treatments.

2.4 Average Length of Stay (LOS)

```
Avg LOS = AVERAGE(HealthData[Length of Stay])
```

Length of Stay (LOS) is a key efficiency metric for hospitals. A high LOS means patients are staying longer — which may indicate complex cases, slow recovery, or operational issues. A low LOS often means efficient treatment. This formula gives the average number of days patients spent in the hospital.

Real-Life Case Study

If 5 patients stayed 3, 5, 7, 2, and 8 days, the Average LOS = (3+5+7+2+8)/5 = 5 days. Hospital managers compare this to national benchmarks. If the national average is 4 days and yours is 7, it signals a need to investigate.

2.5 Doctors

```
Doctors = DISTINCTCOUNT(HealthData[Doctor])
```

Counts the number of unique doctors who have treated patients in the selected time period. This is used to measure workforce engagement — are the same few doctors handling all patients, or is the load well distributed?

2.6 Rooms

```
Rooms = DISTINCTCOUNT(HealthData[Room Number])
```

Counts how many unique hospital rooms have been occupied. A high room utilization rate means the hospital is operating close to capacity — important for expansion planning and resource allocation.

2.7 Total Billing

```
Total Billing = SUM(HealthData[Billing Amount])
```

SUM adds up every billing amount across all patients. This is the total revenue generated by the hospital in the selected time period. Unlike AVERAGE, which shows the per-patient amount, SUM shows the grand total.

SUM vs AVERAGE — When to Use Which?

Use SUM when you want TOTAL revenue (e.g., "How much did we earn this quarter?"). Use AVERAGE when you want a TYPICAL value (e.g., "How much does the average patient pay?"). Both are needed for different insights.

SECTION 3: TEST RESULT ANALYSIS

Medical test results are categorized as Normal, Abnormal, or Inconclusive. These formulas calculate the count and percentage for each category, enabling doctors and administrators to track outcome distributions.

3.1 Abnormal Results — Count

```
Abnormal Results = CALCULATE([Admitted Patients], HealthData[Test Results] =
"Abnormal") +0
```

Why CALCULATE?

CALCULATE is the most powerful function in DAX. It evaluates a measure (like [Admitted Patients]) but applies a FILTER to change the context. Without CALCULATE, [Admitted Patients] would count ALL patients. With CALCULATE, it only counts patients where Test Results = 'Abnormal'.

Syntax

CALCULATE(<expression>, <filter1>, <filter2>, ...) — evaluates the expression in a modified filter context. The "+0" at the end ensures the measure returns 0 instead of BLANK when there are no abnormal results.

Real-Life Case Study

Your dashboard has a filter applied for "Dr. Smith" and "March 2024". Without CALCULATE, every measure uses that filtered context. CALCULATE lets you say: "Within this filtered view, count only the Abnormal cases." This is like a WHERE clause in SQL.

3.2 Abnormal Results — Percentage

```
AbnormalR % = DIVIDE([Abnormal Results], [Admitted Patients], 0) +0
```

Why DIVIDE instead of just dividing with /?

In DAX, if you write [Abnormal Results] / [Admitted Patients] and the denominator is zero (no patients selected), Power BI throws an error or shows BLANK. The DIVIDE function gracefully handles this by returning a default value (0 in this case) instead of an error.

Syntax

DIVIDE(<numerator>, <denominator>, <alternate_result>) — divides two numbers safely. The third argument is what to return if the denominator is zero.

Real-Life Case Study

If 40 patients have abnormal results out of 200 total, the formula returns 40/200 = 0.20, displayed as 20% on the dashboard. If a filter results in 0 patients, DIVIDE returns 0 instead of breaking the visual.

3.3 Normal and Inconclusive Results

```
Normal Results = CALCULATE([Admitted Patients], HealthData[Test Results] =
"Normal") +0
Normal Result % = DIVIDE([Normal Results],[Admitted Patients],0) +0


Inconclusive Results = CALCULATE([Admitted Patients], HealthData[Test Results]
= "Inconclusive") +0
Inconclusive % = DIVIDE([Inconclusive Results], [Admitted Patients],0)+0
```

These use the same pattern as Abnormal Results, just filtering for 'Normal' or 'Inconclusive' test results. Together, these three pairs of measures give a complete picture of diagnostic outcomes, allowing managers to track whether test result distributions are changing over time.

SECTION 4: DYNAMIC TITLES

Static titles don't change when users apply filters. Dynamic titles update automatically based on what the user has selected, making the dashboard much more intuitive and professional.

4.1 Line Chart Title

```
Line Title =

SWITCH(

    SELECTEDVALUE('KPI'[KPI Order]),

    0, "Currently Viewing Admitted Patients Trend",

    1, "Currently Viewing Avg Length of Stay Trend",

    "Currently Viewing Avg Billing Trend"

)
```

How Does SWITCH Work?

SWITCH is like a multi-option IF statement. It checks the value of SELECTEDVALUE('KPI'[KPI Order]) and returns different text based on which KPI the user has selected.

Syntax

SWITCH(<expression>, <value1>, <result1>, <value2>, <result2>, ..., <else_result>) — checks the expression against each value in order and returns the matching result. If no value matches, the else_result is returned.

KPI Order Value	Title Shown on Dashboard
`0`	Currently Viewing Admitted Patients Trend
`1`	Currently Viewing Avg Length of Stay Trend
`Any other value`	Currently Viewing Avg Billing Trend

Real-Life Case Study

Imagine a dashboard with a toggle button that lets users switch between viewing "Admitted Patients," "Avg LOS," and "Avg Billing" on the same line chart. The Line Title formula detects which option is selected and updates the chart title automatically — no manual editing required.

4.2 Billing Section Title

```
Title =

Var MedCon = SELECTEDVALUE(DimCondition[Medical Condition])

RETURN

"Total Billing Accumulated by" &" "& MedCon
```

This formula reads the currently selected medical condition (e.g., Diabetes, Cancer, Arthritis) from a slicer and builds a sentence like: 'Total Billing Accumulated by Diabetes'.

Key Functions Used

SELECTEDVALUE returns the single selected value from a column — if multiple values are selected, it returns BLANK. The & operator joins (concatenates) text strings together.

VAR...RETURN is a clean way to store an intermediate calculation in a variable before using it.

SECTION 5: YEAR-OVER-YEAR (YoY) FORMULAS

Year-over-Year (YoY) comparisons are the backbone of performance analysis. They answer the question: 'Compared to last year, are things better or worse?' All 6 YoY formulas in this dashboard follow the exact same pattern — only the measure being compared changes.

The YoY Pattern (used in all 6 formulas)

1. Get the currently selected year from the Calendar slicer. 2. Calculate the previous year (selected year minus 1). 3. Calculate the measure value for the previous year. 4. Calculate the change percentage. 5. Format it with an up/down arrow and display it.

5.1 Admitted Patients YoY

```
Adm YoY =

VAR SelectedYear = SELECTEDVALUE('Calendar'[Year])

VAR PrevYear =

    IF(NOT ISBLANK(SelectedYear),

        SelectedYear - 1,

        BLANK()

    )

VAR PrevYearValue =
```

```
    IF(NOT ISBLANK(PrevYear),
        CALCULATE([Admitted Patients], YEAR('Calendar'[Date]) = PrevYear),
        BLANK()
    )
VAR CurrentValue = [Admitted Patients]
VAR YoYChange =
    IF(
        AND(NOT ISBLANK(CurrentValue), NOT ISBLANK(PrevYearValue)),
        DIVIDE(CurrentValue - PrevYearValue, PrevYearValue),
        BLANK()
    )
RETURN
IF(
    ISBLANK(SelectedYear),
    "No selection",
    IF(
        NOT ISBLANK(YoYChange),
        IF(YoYChange > 0,
            "↑ " & FORMAT(YoYChange, "#.0%") & " Vs " & PrevYear,
            "↓ " & FORMAT(YoYChange, "#.0%") & " Vs " & PrevYear
        ),
        "No Data"
    )
)
```

Detailed Breakdown — Variable by Variable

Variable / Part	What It Does and Why
`VAR SelectedYear`	Stores the year the user has selected in the slicer (e.g., 2023). Using VAR keeps the formula clean and avoids repeating the same expression.
`VAR PrevYear`	Calculates last year: 2023 - 1 = 2022. If no year is selected, it stores BLANK to avoid errors.
`VAR PrevYearValue`	Uses CALCULATE to compute [Admitted Patients] but filtered to only the previous year's dates. This is how we get the 'last year' number.
`VAR CurrentValue`	Holds the current year's admitted patients count (already filtered by the slicer).
`VAR YoYChange`	Divides (Current - Previous) by Previous to get the % change. AND checks both values exist before dividing.
`NOT ISBLANK()`	A safety check — if either value is missing, skip the calculation to avoid errors or misleading results.
`FORMAT(YoYChange, "#.0%")`	Converts a decimal like 0.125 into a readable '12.5%' format.
`"↑" or "↓"`	Adds a visual arrow to indicate whether the metric went up or down compared to last year.

`"Vs " & PrevYear`	Creates the comparison label like "Vs 2022" so the user knows what year is being compared.

Real-Life Case Study

The year slicer is set to 2024. [Admitted Patients] returns 1,200 (current year). PrevYearValue calculates to 1,000 (year 2023). YoYChange = (1200-1000)/1000 = 0.20 = 20%. The formula displays: "↑ 20.0% Vs 2023" — meaning admissions grew 20% compared to last year.

5.2 Average Age YoY (Age YoY)

Follows the identical pattern as Adm YoY, but compares [Average age] instead of [Admitted Patients]. This helps track whether the patient demographic is aging over time — which has direct implications for care planning and resource needs.

5.3 Average Billing YoY (Bill YoY)

Compares [Avg Billing Amt] year-over-year. Rising billing amounts may indicate higher acuity cases, inflationary pricing, or longer stays. Falling amounts could signal efficiency improvements or a shift to outpatient care.

5.4 Doctors YoY (Doc YoY)

Tracks whether the number of active doctors has increased or decreased compared to the prior year. A drop in doctors while patient volume rises is a red flag for staffing stress. An increase may reflect expansion or specialization growth.

5.5 Length of Stay YoY (LOS YoY)

Compares average length of stay year-over-year. Hospitals aim to reduce LOS over time — shorter stays generally mean more efficient care and lower costs. A year-over-year increase in LOS warrants investigation.

5.6 Rooms YoY (Room YoY)

Tracks changes in the number of occupied rooms year-over-year. Combined with patient count data, this helps understand whether rooms are being used more or less efficiently across years.

SECTION 6: DAX FUNCTIONS QUICK REFERENCE

This table summarizes every DAX function used in this dashboard for quick reference.

Function	Category	What It Does
`DISTINCTCOUNT`	*Aggregation*	Counts unique values in a column — ignores duplicates.
`AVERAGE`	*Aggregation*	Calculates the mean (sum divided by count) of a numeric column.
`SUM`	*Aggregation*	Adds up all values in a numeric column.
`CALCULATE`	*Filter*	Re-evaluates an expression after applying one or more filters.
`DIVIDE`	*Math*	Divides two numbers safely, returning a default if divisor is zero.
`CALENDAR`	*Date/Time*	Creates a single-column table with one row for each date in a range.
`ADDCOLUMNS`	*Table*	Adds calculated columns to a table.
`YEAR / MONTH / DAY`	*Date/Time*	Extracts the year, month number, or day number from a date.
`QUARTER`	*Date/Time*	Returns the quarter number (1-4) for a given date.

WEEKNUM / WEEKDAY	*Date/Time*	Returns the week number of the year, or day of the week.
FORMAT	*Text*	Converts a value to text using a specified format pattern.
SWITCH	*Logical*	Evaluates an expression and returns a result from multiple options.
SELECTEDVALUE	*Filter*	Returns the single selected value from a column in filter context.
ISBLANK	*Information*	Returns TRUE if a value is BLANK (empty/missing).
IF	*Logical*	Returns one value if a condition is true, another if false.
AND	*Logical*	Returns TRUE only if all conditions are TRUE.
VAR / RETURN	*Variables*	Stores intermediate results in named variables for cleaner formulas.
MIN / MAX	*Aggregation*	Returns the minimum or maximum value in a column.

SECTION 7: BEST PRACTICES FOR BEGINNERS

7.1 Tips for Writing Clean DAX

- Always use VAR to store repeated expressions — it makes formulas readable and avoids re-calculating the same value multiple times.
- Use DIVIDE instead of the / operator to handle division-by-zero errors gracefully.
- Add +0 to CALCULATE measures to prevent BLANK from appearing in visuals — return 0 instead.
- Use DISTINCTCOUNT for unique counts (patients, doctors, rooms) and COUNT for total rows.
- Always create a Calendar Table before building date-based calculations.
- Format percentages using FORMAT(<value>, '#.0%') for consistent display.
- Test each measure individually before combining them into complex formulas.

7.2 Common Mistakes to Avoid

- Using COUNT instead of DISTINCTCOUNT — COUNT counts every row, DISTINCTCOUNT counts unique values.
- Forgetting to handle BLANK values — many DAX functions return BLANK instead of 0, which can break visuals.
- Not creating a Calendar Table — date intelligence functions (like SAMEPERIODLASTYEAR) require it.

- Writing long formulas without VAR — variables make formulas much easier to debug and maintain.
- Using / for division without DIVIDE — if the denominator is ever 0, this will cause an error.

7.3 How the Formulas Connect

Understanding how these formulas depend on each other helps you debug and extend the dashboard:

Dependency Chain

Calendar Table → enables all YoY calculations [Admitted Patients] → used inside all YoY formulas as the base measure [Abnormal Results] → uses [Admitted Patients] as denominator in AbnormalR% SELECTEDVALUE → powers both dynamic titles and YoY year selection CALCULATE → the engine behind all filtered measures